

Synchronization

Mata Kuliah Sistem Operasi

Dosen Pengampu: Triawan Adi Cahyanto, M.Kom

Program Studi: Teknik Informatika

Universitas Muhammadiyah Jember

2025

Background

🔄 Apa itu Sinkronisasi?

Mekanisme untuk **mengkoordinasi** eksekusi beberapa proses yang mengakses sumber daya bersama

! Mengapa Penting?

- Mencegah **race condition**
- Menjaga **konsistensi data**
- Menghindari **deadlock**
- Memastikan **eksekusi yang benar**

💡 Contoh Sederhana

Dua orang mencoba menulis di papan tulis yang sama secara bersamaan. Tanpa sinkronisasi, tulisan mereka bisa saling menimpa atau tidak terbaca dengan jelas.

Sinkronisasi = Antrian Teratur

⚠️ Masalah Tanpa Sinkronisasi



Race Condition



Deadlock



Inkonsistensi Data



Starvation

✅ Tujuan Sinkronisasi

Memastikan **hanya satu proses** yang dapat mengakses sumber daya bersama pada satu waktu

Analogi: **Toilet umum** dengan satu pintu - hanya satu orang yang bisa masuk pada satu waktu

The Critical-Section Problem



<> Apa itu Critical Section?

Bagian program di mana **sumber daya bersama** diakses oleh beberapa proses

⚠ Mengapa Menjadi Masalah?

- **Race condition** - hasil tidak dapat diprediksi
- **Inkonsistensi data** - data rusak
- **Kerugian performa** - proses saling menunggu

💡 Contoh Sederhana

Dua orang mencoba menarik uang dari rekening bersama yang memiliki saldo Rp 1.000.000. Keduanya mencoba menarik Rp 800.000 secara bersamaan.

Tanpa sinkronisasi, keduanya bisa berhasil menarik uang!

✅ Persyaratan Solusi



Mutual Exclusion

Hanya satu proses di critical section pada satu waktu



Progress

Proses di luar critical section tidak memblokir proses lain



Bounded Waiting

Ada batas waktu tunggu untuk masuk ke critical section



Analogi

Toilet umum dengan satu pintu:

- Hanya satu orang yang bisa masuk
- Orang di luar tidak memblokir pintu
- Setiap orang akan mendapat giliran

Peterson's Solution

Waiter's Problem

Waiter's Problem

Waiter's Problem

Waiter's Problem

Waiter's Problem

Waiter's Problem

Waiter's Problem

Waiter's Problem

Waiter's Problem

💡 Apa itu Peterson's Solution?

Solusi **berbasis perangkat lunak** untuk masalah critical section yang hanya berfungsi untuk **dua proses**

⚙️ Bagaimana Cara Kerjanya?

- Proses menandakan **keinginan masuk** ke critical section
- Proses **memberikan giliran** ke proses lain
- Proses menunggu hingga mendapat giliran
- Hanya satu proses yang bisa masuk pada satu waktu



flag[2]

Menandakan apakah proses ingin masuk ke critical section



turn

Menentukan proses mana yang mendapat giliran

👍 Kelebihan

- Tidak memerlukan perangkat keras khusus
- Sederhana dan mudah dipahami
- Memenuhi semua persyaratan

🗨️ Kekurangan

- Hanya untuk dua proses
- Busy waiting (memakan CPU)
- Tidak efisien untuk sistem modern

⌘ Algoritma Peterson's Solution

```
// Variabel bersama boolean flag[2] = {false, false}; int turn; // Proses i (0 atau 1) do { // Menandakan keinginan masuk flag[i] = true; // Memberikan giliran ke proses lain turn = j; // Menunggu hingga mendapat giliran while (flag[j] && turn == j); // Critical Section // ... kode di sini ... // Keluar dari critical section flag[i] = false; // Remainder Section // ... kode di sini ... } while (true);
```

🧠 Analogi Sederhana

Dua orang ingin menggunakan **toilet umum** dengan satu pintu:

- Setiap orang **meletakkan kunci** di kotak saat ingin masuk
- Setiap orang **memberikan giliran** ke orang lain
- Orang hanya bisa masuk jika kuncinya ada di kotak dan gilirannya tiba

"Saling menghormati giliran untuk mencegah konflik"

Hardware Support for Synchronization

🔧 Mengapa Perlu Dukungan Perangkat Keras?

- Solusi perangkat lunak **kompleks**
- **Busy waiting** memboroskan CPU
- Dukungan perangkat keras **lebih efisien**
- Menyediakan operasi **atomik**

➡ Implementasi Mutex dengan Test and Set

Menggunakan Test and Set untuk implementasi lock:

```
// Variabel bersama boolean lock = false; //  
Proses while (TestAndSet(&lock)) ; // busy  
wait // Critical Section // ... kode di sini  
... // Unlock lock = false;
```

Hanya satu proses bisa melewati while loop

✅ Test and Set

Menguji dan mengatur nilai atomik

```
boolean  
TestAndSet(boolean  
*target) { boolean rv =  
*target; *target =  
true; return rv; }
```

↔ Swap

Menukar nilai dua variabel atomik

```
void Swap(boolean *a,  
boolean *b) { boolean  
temp = *a; *a = *b; *b  
= temp; }
```

👍 Kelebihan

- Operasi atomik
- Sederhana
- Efisien
- Didukung CPU modern

🗨 Kekurangan

- Busy waiting
- Prioritas inversion
- Platform dependent

🧠 Analogi

Pintu dengan sistem kunci elektronik:

- Hanya satu kartu yang bisa membuka pada satu waktu
- Sistem memastikan tidak ada dua orang yang masuk bersamaan
- Kunci otomatis terkunci setelah pintu ditutup

🔒 Unlock and Lock

Mengunci dan membuka akses ke sumber daya

🔧 Memory Barriers

Memastikan urutan operasi memori

Mutex Locks

Apa itu Mutex Locks?

Mekanisme **penguncian** untuk sinkronisasi akses ke sumber daya bersama

Mutual Exclusion = Hanya satu proses pada satu waktu

lock()

Mengunci mutex sebelum memasuki critical section

```
acquire() { while  
(!available) ; // busy  
wait available = false;  
}
```

unlock()

Membuka mutex setelah keluar dari critical section

```
release() { available =  
true; }
```

Kelebihan

- Sederhana
- Memastikan mutual exclusion
- Portabel

Kekurangan

- Busy waiting
- Deadlock jika tidak hati-hati
- Priority inversion

Cara Kerja Mutex

- Mutex memiliki status **terkunci/terbuka**
- Proses **meminta kunci** sebelum akses
- Jika kunci **tersedia**, proses dapat masuk
- Jika kunci **terkunci**, proses menunggu
- Setelah selesai, proses **melepaskan kunci**

<> Contoh Implementasi

```
// Variabel mutex mutex lock; // Proses  
lock.acquire(); // Critical Section // ...  
akses sumber daya bersama ... lock.release();  
// Remainder Section // ... kode lain ...
```

Analogi

Kamar mandi umum dengan satu kunci:

- Hanya ada **satu kunci** untuk semua orang
- Orang harus **meminjam kunci** sebelum masuk
- Jika kunci sedang dipinjam, orang lain harus **menunggu**
- Setelah selesai, kunci **dikembalikan**

Semaphores

🔧 Apa itu Semaphores?

Mekanisme sinkronisasi yang **lebih fleksibel** dari mutex

Variabel integer + operasi atomik

🔛 Binary

Nilai 0 atau 1

Mirip dengan mutex

1 Counting

Nilai non-negatif

Mengontrol akses ke beberapa sumber daya

↔ Operasi Dasar

⌚ wait() / P()

Mengurangi nilai semaphore

```
wait(S) { while (S <= 0) ; // busy wait S--; }
```

⊕ signal() / V()

Menambah nilai semaphore

```
signal(S) { S++; }
```

⚙ Cara Kerja Semaphores

- Semaphore memiliki **nilai integer**
- Proses memanggil **wait()** sebelum akses
- Jika nilai > 0, proses **dapat melanjutkan**
- Jika nilai = 0, proses **menunggu**
- Setelah selesai, proses memanggil **signal()**

⌌ Contoh Implementasi

```
// Variabel semaphore semaphore S = 1; //  
Proses wait(S); // Critical Section // ...  
akses sumber daya bersama ... signal(S); //  
Remainder Section // ... kode lain ...
```

👍 Kelebihan

- Fleksibel
- Dapat mengontrol beberapa sumber daya
- Menyelesaikan masalah sinkronisasi kompleks

🗨 Kekurangan

- Busy waiting
- Deadlock jika tidak hati-hati
- Lebih kompleks dari mutex

Monitors

Apa itu Monitors?

Konstruksi sinkronisasi **tingkat tinggi** yang menggabungkan data dan prosedur dalam satu unit

Mutual exclusion otomatis

Struktur Monitor



Mutual Exclusion - Hanya satu proses dalam monitor pada satu waktu



Data Privat - Hanya dapat diakses melalui prosedur monitor



Prosedur - Fungsi yang dapat dipanggil proses eksternal



Kondisi Variabel - Untuk sinkronisasi kondisional



wait()

Menangguhkan eksekusi proses hingga kondisi terpenuhi



signal()

Membangunkan salah satu proses yang menunggu

<> Contoh Implementasi

```
monitor ResourceMonitor { boolean busy = false; condition x; void acquire() { if (busy) x.wait(); busy = true; } void release() { busy = false; x.signal(); } }
```



Kelebihan

- Otomatis mutual exclusion
- Tingkat abstraksi tinggi
- Mencegah kesalahan programmer



Kekurangan

- Tidak semua bahasa mendukung
- Implementasi kompleks
- Performa lebih lambat



Analogi

Kantor bank dengan teller:

- Hanya **satu nasabah** dilayani teller pada satu waktu
- Nasabah **menunggu** di ruang tunggu jika teller sibuk
- Teller **memanggil** nasabah berikutnya saat selesai
- Nasabah tidak bisa **mengakses** uang langsung

Liveness

🕒 Apa itu Liveness?

Jaminan bahwa **beberapa tindakan akan terjadi** pada akhirnya dalam sistem konkuren

Sistem tidak "macet" selamanya

🔧 Solusi Masalah Liveness

🕒 **Timeout** - Batas waktu tunggu

! **Priority Inheritance** - Naikkan prioritas

↔ **Resource Ordering** - Urutan akses tetap

🔄 **Preemption** - Paksa lepas sumber daya

💡 Contoh Masalah Liveness

Deadlock: Dua mobil di jalan sempit saling menunggu untuk lewat

Starvation: Anak pendiam selalu kalah dalam permainan rebutan

Priority Inversion: Dokter ahli menunggu perawat membersihkan ruangan

🚫 Deadlock

Proses saling menunggu sumber daya yang dipegang proses lain



Starvation

Proses tidak pernah mendapat kesempatan untuk menjalankan tugasnya



Priority Inversion

Proses prioritas tinggi menunggu proses prioritas rendah

🧠 Analogi

Restoran dengan meja terbatas:

- **Deadlock**: Dua kelompok saling menunggu meja yang sama
- **Starvation**: Pelanggan tunggal tidak pernah dapat meja
- **Priority Inversion**: VIP menunggu pelanggan biasa selesai

Evaluation

Kriteria Evaluasi

- **Performa** - Kecepatan eksekusi
- **Efisiensi** - Penggunaan sumber daya
- **Keandalan** - Kemampuan mencegah error
- **Skalabilitas** - Performa dengan beban tinggi
- **Kemudahan** - Tingkat kesulitan implementasi

Perbandingan Mekanisme

Mekanisme	Performa	Kompleksitas
Mutex	★★★★☆	★★★☆☆
Semaphore	★★★★☆	★★★★☆
Monitor	★★★★☆	★★★★★

Studi Kasus

Database Sistem:

- **Mutex** untuk kontrol akses tabel
- **Semaphore** untuk pool koneksi
- **Monitor** untuk transaksi kompleks

Throughput

Jumlah operasi per satuan waktu

Latency

Waktu respons untuk satu operasi

Fairness

Distribusi sumber daya yang adil

Scalability

Performa dengan peningkatan beban

Kesimpulan

- **Tidak ada solusi sempurna** untuk semua kasus
- Pilih mekanisme sesuai **kebutuhan spesifik**
- Pertimbangkan **trade-off** antara performa dan kompleksitas

Summary

Konsep Sinkronisasi

- **Koordinasi** proses yang mengakses sumber daya bersama
- Mencegah **race condition** dan **deadlock**
- Memastikan **konsistensi data**
- Tiga persyaratan: **mutual exclusion**, **progress**, **bounded waiting**

Mutex

Sederhana, mutual exclusion, hanya satu proses

Semaphore

Fleksibel, mengontrol beberapa sumber daya

Monitor

Tingkat tinggi, otomatis mutual exclusion

Hardware

Operasi atomik, efisien, platform dependent

Kapan Menggunakan?

 **Mutex:** Akses tunggal ke sumber daya

 **Semaphore:** Beberapa sumber daya identik

 **Monitor:** Sinkronisasi kompleks

 **Hardware:** Performa tinggi

Poin Penting

- Tidak ada solusi sempurna untuk semua kasus
- Pertimbangkan trade-off antara performa dan kompleksitas
- Perhatikan masalah liveness (deadlock, starvation)
- Pilih mekanisme sesuai kebutuhan spesifik aplikasi
- Implementasi yang benar sama pentingnya dengan pemilihan mekanisme

Kesimpulan

Sinkronisasi adalah **fondasi** sistem operasi modern. Memahami berbagai mekanisme dan kapan menggunakannya adalah **keterampilan penting** untuk pengembang sistem.

Classic Problems of Synchronization



Producer-Consumer Problem

Producer menambahkan item ke buffer, Consumer mengambil item dari buffer



Readers-Writers Problem

Banyak Readers bisa mengakses bersamaan, Writer harus eksklusif



Dining Philosophers Problem

5 filsuf membutuhkan 2 sumpit untuk makan, deadlock jika semua mengambil 1 sumpit



Solusi



Producer-Consumer: 2 semaphores (empty & full) + mutex untuk buffer



Readers-Writers: Semaphore untuk readers, mutex untuk writers, atau solusi dengan prioritas



Dining Philosophers: Resource hierarchy, arbitrator, atau Chandy-Misra algorithm



Relevansi dengan Sistem Modern

- **Producer-Consumer:** Message queue, buffer pool, task scheduling
- **Readers-Writers:** Database access, file system, cache management
- **Dining Philosophers:** Resource allocation, deadlock prevention



Analogi Sederhana

Producer-Consumer: Restoran dengan dapur dan pelayan

Readers-Writers: Perpustakaan dengan pembaca dan penulis

Dining Philosophers: Keluarga makan bersama dengan alat makan terbatas



Mengapa Penting?

- **Model abstrak** untuk masalah sinkronisasi nyata
- **Basis pengembangan** algoritma sinkronisasi modern
- **Latihan praktis** untuk memahami konsep deadlock dan starvation

Masalah klasik = Fondasi sinkronisasi modern

Synchronization within the Kernel

🔧 Mengapa Penting dalam Kernel?

- Kernel mengelola **sumber daya bersama**
- **Multi-core processing** memerlukan sinkronisasi
- Mencegah **race condition** pada level sistem
- Menjaga **konsistensi data struktur** kernel

📁 Sinkronisasi Level CPU

- **Interrupt disabling** - Mencegah interrupt
- **Preemption disabling** - Mencegah context switch
- **Memory barriers** - Urutan operasi memori
- **Atomic operations** - Operasi tidak terputus

🔗 Implementasi Linux Kernel

Spinlock untuk critical section singkat:

- Scheduler, timer, dan interrupt handling
- Struktur data yang sering diakses

Mutex untuk critical section panjang:

- File system operations
- Device driver interactions

RCU untuk read-heavy:

- Network routing tables
- Process management structures

🔒 Spinlocks

Busy waiting untuk critical section singkat

🚫 Mutexes

Blocking lock untuk critical section panjang

🚦 Semaphores

Mengontrol akses ke beberapa sumber daya

🔄 RCU

Read-Copy-Update untuk read-heavy workloads

⚠️ Tantangan

- **Performance overhead** dari sinkronisasi
- **Deadlock** antar komponen kernel
- **Priority inversion** pada real-time systems
- **Scalability** dengan peningkatan core CPU
- **Debugging** masalah sinkronisasi yang kompleks

POSIX Synchronization

⚙️ Apa itu POSIX?

Portable Operating System Interface - Standar untuk API sistem operasi

Memastikan kompatibilitas antar platform

👤 POSIX Threads (pthreads)

- Standar untuk **thread programming**
- Mendukung **multi-threading** di berbagai OS
- Menyediakan API untuk **sinkronisasi thread**

🔒 Mutexes

Mutual exclusion untuk critical section

🚦 Semaphores

Mengontrol akses ke beberapa sumber daya

🔔 Condition Variables

Sinkronisasi berdasarkan kondisi

👉 Thread Joining

Menunggu thread selesai eksekusi

⏏ Contoh Implementasi

```
// Inisialisasi mutex pthread_mutex_t mutex;
pthread_mutex_init(&mutex, NULL); // Thread
function void* thread_function(void* arg) {
// Lock mutex pthread_mutex_lock(&mutex); //
Critical Section // ... akses sumber daya
bersama ... // Unlock mutex
pthread_mutex_unlock(&mutex); return NULL; }
// Main function int main() { pthread_t
thread_id; // Create thread
pthread_create(&thread_id, NULL,
thread_function, NULL); // Wait for thread to
finish pthread_join(thread_id, NULL); //
Destroy mutex pthread_mutex_destroy(&mutex);
return 0; }
```

👍 Kelebihan

- Portabel antar platform
- Standar industri
- Dokumentasi lengkap
- Banyak contoh implementasi

🗨 Kekurangan

- Sintaks lebih kompleks
- Error handling manual
- Resource management manual
- Tidak thread-safe default

🧠 Analogi

POSIX Synchronization seperti **peraturan lalu lintas universal**:

- Sama di **semua negara** (platform)
- **Rambu** yang jelas (mutex, semaphore)
- **Lampu lalu lintas** (condition variables)
- **Jalur khusus** untuk gabungan (thread joining)

Synchronization in Java

🔄 Konsep Sinkronisasi dalam Java

- Mekanisme untuk **mengontrol akses** thread ke sumber daya bersama
- Mencegah **race condition** dan **inkonsistensi data**
- Built-in ke dalam bahasa Java

<> synchronized Methods

Method hanya dapat diakses satu thread pada satu waktu

```
public synchronized  
void method() {  
    // critical section  
}
```

🏠 synchronized Blocks

Hanya bagian tertentu method yang disinkronkan

```
synchronized(object) {  
    // critical section  
}
```

🛡️ Mencegah Masalah Thread

- **Thread Interference** - Thread mengubah data bersama secara bersamaan
- **Memory Consistency Errors** - Thread melihat data yang tidak konsisten
- **Happens-Before Relationship** - Jaminan urutan operasi memori

<> Contoh Implementasi

```
public class Counter {  
    private int count = 0;  
    // Synchronized method  
    public synchronized  
    void increment() { count++; }  
    // Synchronized block  
    public void decrement() {  
        synchronized(this) { count--; }  
    }  
    public int getCount() { return count; }  
} // Penggunaan  
Counter counter = new Counter();  
// Thread 1  
new Thread(() -> { for(int i = 0; i < 1000; i++) { counter.increment(); } }).start();  
// Thread 2  
new Thread(() -> { for(int i = 0; i < 1000; i++) { counter.decrement(); } }).start();
```

👍 Kelebihan

- Sintaks sederhana
- Built-in ke Java
- Otomatis lock/unlock
- Prevents race conditions

🗨️ Kekurangan

- Performa lebih lambat
- Risiko deadlock
- Tidak bisa timeout
- Lock granularity kasar

🧠 Analogi

synchronized seperti **ruang meeting**:

- Hanya **satu tim** bisa menggunakan pada satu waktu
- **Pintu terkunci** saat meeting berlangsung
- Tim lain harus **menunggu** di luar
- Pintu **otomatis terbuka** saat meeting selesai

Alternative Approaches



Non-blocking

Thread tidak pernah diblokir, selalu ada kemajuan dalam sistem



Lock-free Algorithms

Algoritma yang tidak menggunakan lock untuk sinkronisasi



Transactional Memory

Eksekusi transaksi atomik, rollback jika konflik

👍 Kelebihan

- Tidak ada deadlock
- Performa lebih baik
- Skalabilitas tinggi
- Responsifitas sistem

🗨️ Kekurangan

- Implementasi kompleks
- Debugging sulit
- Tidak semua platform mendukung
- Memory overhead



Kapan Menggunakan?

- **High-performance systems** dengan beban tinggi
- **Real-time systems** yang membutuhkan respons cepat
- **Multi-core systems** dengan kontensi tinggi
- Sistem yang **tidak toleran terhadap blocking**

<> Contoh Implementasi

Lock-free Counter dengan Compare-and-Swap:

Transactional Memory untuk database:

Non-blocking Queue untuk producer-consumer:



Analogi

Non-blocking seperti **jalan tol tanpa gerbang**:

- Kendaraan **tidak berhenti** di gerbang
- **Tidak ada antrian** yang menyebabkan kemacetan
- Sistem **elektronik** mendeteksi dan menagih otomatis
- Konflik **diselesaikan** tanpa menghentikan alur

Summary

Ringkasan Contoh Sinkronisasi

- **Masalah klasik** (Producer-Consumer, Readers-Writers, Dining Philosophers)
- **Implementasi kernel** dengan spinlocks, mutexes, RCU
- **Standar POSIX** dengan pthreads, mutexes, semaphores
- **Java synchronization** dengan synchronized keyword
- **Pendekatan alternatif** non-blocking dan lock-free

Kernel Level

Performa tinggi, kompleksitas tinggi

POSIX

Portabel, standar industri

Java

Sederhana, built-in ke bahasa

Non-blocking

Performa optimal, implementasi sulit

Poin Penting

- Pilih mekanisme sinkronisasi sesuai **kebutuhan aplikasi**
- Pertimbangkan **trade-off** antara performa dan kompleksitas
- Perhatikan masalah **liveness** (deadlock, starvation)
- Sinkronisasi adalah **fondasi** sistem konkuren modern
- Tidak ada solusi **sempurna** untuk semua kasus

Kesimpulan

Sinkronisasi adalah **mekanisme krusial** dalam sistem operasi modern. Memahami berbagai pendekatan dan implementasinya memungkinkan pengembang untuk **membangun sistem** yang efisien, andal, dan skalabel.



Terima Kasih

Pertanyaan & Diskusi